

Architecting a Fault-Tolerant Video Persistence Pipeline for BreakDex

Executive Summary

The "add move" feature within the BreakDex application is currently afflicted by a series of critical data integrity and user experience failures. Analysis reveals that the existing implementation, which processes and saves user-edited training footage, is architecturally fragile. It fails to reliably persist user-applied video edits, such as trimming and rotation, leading to a discrepancy between the user's preview and the final saved asset. Furthermore, it improperly manages interactions with the system's Photos library, resulting in the creation of duplicate or empty albums. The persistence layer in CoreData is also flawed, storing unstable references that break the link between the application's data and the underlying media asset. These issues collectively undermine the user's trust and violate the core requirement for a precise, reliable tool, akin to a "mechanical watch."

This report presents a comprehensive diagnosis of these failures and proposes a new, robust architectural solution. The analysis deconstructs the system from first principles, modeling the data flow as a series of state transitions to pinpoint specific points of failure, such as non-atomic operations, incorrect geometric transformations, and a misunderstanding of data ownership within the iOS ecosystem.

The proposed solution replaces the current fragile, sequential process with a fault-tolerant, transactional, and state-managed pipeline. This is achieved through the introduction of a central VideoPersistenceManager that orchestrates the entire workflow as a single, resilient operation. The new architecture guarantees a WYSIWYG (What You See Is What You Get) experience by correctly applying video transformations. It ensures data integrity through atomic interactions with the PhotoKit and CoreData frameworks and builds in resilience to environmental factors such as iCloud-hosted assets and application backgrounding. The implementation of this architecture will rectify all identified issues, delivering the frictionless and mechanically precise experience that the application's users demand.

I. A Categorical Model of the Video Persistence Pipeline

To formally reason about the system's correctness and diagnose its failures with precision, it is instructive to model the video persistence pipeline using concepts from category theory. In this model, the various states of the video data are treated as **objects**, and the operations that transform the data from one state to another are treated as **morphisms**. The ideal, error-free system can then be represented as a commutative diagram, where the composition of morphisms from an initial state to a final state is deterministic and yields a single, correct result, regardless of the specific execution path.

1.1. Defining the System Objects (States of Data)

The lifecycle of a video asset, from selection to final persistence, can be described by a sequence of distinct data states. Each state represents a specific form and location of the data.

- **SourcePHAsset:** The initial object in our system. This represents the immutable video asset selected by the user from the iOS Photos library, encapsulated by a PHAsset instance. This object serves as the ground truth for the original media. It is critical to recognize that this asset may exist fully on the local device or may be a placeholder for content stored exclusively in iCloud Photos.¹
- **EditingState:** A transient data object that captures the user's editing intent. This object is not a media asset itself but a collection of parameters. It contains the desired trim range, represented by a CMTimeRange, and the desired rotation, represented by a rotation angle (e.g., 0, 90, 180, or 270 degrees). This object represents the user's desired final state for the video's content and orientation.
- **LocalAVAsset:** An intermediate object representing the video asset's data made fully available on the local device as an AVAsset. The transition to this state is non-trivial; it requires resolving the SourcePHAsset, which may involve an asynchronous download from iCloud if the asset is not stored locally. This object is the direct input for any media processing operations.
- **ProcessedVideoFile:** The tangible result of applying the EditingState to the LocalAVAsset. This object is a temporary video file (e.g., in .mp4 or .mov format) stored in the application's local cache or temporary directory. This file contains the video data with the trim and rotation transformations physically applied to the media stream.
- **LibraryPHAsset:** The new, permanent asset created within the user's Photos library. This object is the ultimate "source of truth" for the saved media clip. It is created by saving the

ProcessedVideoFile into a designated "BreakDex" album. Its existence and properties are managed by the system's PhotoKit framework.

- **CoreDataRecord:** The final object in the persistence pipeline. This is an NSObject instance within the application's CoreData database. It stores the metadata for the move, such as its name, and, most importantly, a stable, persistent reference that uniquely identifies the corresponding LibraryPHAsset. Storing a reference, rather than the binary data itself, is a recognized best practice for managing large media files with CoreData.²

1.2. Mapping the System Morphisms (Operations and Functions)

The transitions between the objects defined above are accomplished through a series of operations, or morphisms. Each morphism represents a function or a process within the application.

- **PHAssetPicker** : The user action of selecting a video from the system's photo picker interface. This initiates the entire workflow.
- **PHAsset** : This is a composite morphism that represents the user applying edits in the UI and the system's subsequent preparation of the asset for processing. This operation must handle the critical logic of determining if the SourcePHAsset is an iCloud asset and, if so, initiating its download to produce a

LocalAVAsset.

- `AVAssetExportSession` : The core video processing operation. This is typically implemented using `AVAssetExportSession`. It is an asynchronous, potentially long-running, and fallible morphism that takes the local asset and the user's edits and produces a new video file on disk.³
- `PHPhotoLibrary.performChanges` : The transactional operation of adding the processed video file to the user's Photos library. This is performed via `PHPhotoLibrary.performChanges`. A crucial requirement for this morphism is that it must be idempotent with respect to album creation; that is, it must find and use the "BreakDex" album if it exists, and create it only if it does not, without creating duplicates.⁴
- `PHAsset` : The final persistence step. This morphism takes the newly created `LibraryPHAsset` and the user-provided name for the move and creates a corresponding record in the CoreData database. The correctness of this morphism hinges on its ability to store a stable, permanent reference to the `LibraryPHAsset`, such as

its localIdentifier.⁶

1.3. The Ideal Commutative Diagram: A Blueprint for Correctness

In a perfectly functioning system, the sequence of operations forms a clear, deterministic path from the initial user selection to the final persisted state. The overall transformation can be expressed as the composition of the individual morphisms:

The principle of commutativity in this context means that the final state of the system—a CoreDataRecord that correctly points to a LibraryPHAsset which visually matches the user's edits in the EditingState—is guaranteed, regardless of any underlying asynchronicity, retries, or other runtime variations. This formalizes the WYSIWYG requirement. The user's preview of the edits and the final state of the saved asset must be equivalent. Any deviation from this equivalence indicates that the diagram has failed to commute, and a bug exists in the implementation of one or more morphisms or their composition.

II. Analysis of Pipeline Failure: Non-Commutativity and Isomorphism Breakdowns

Using the categorical model as a framework, a rigorous analysis of the application's logs and code reveals multiple points where the implementation deviates from the ideal, leading to the observed failures. Each bug corresponds to a specific breakdown in the model, such as a non-commutative diagram where the order of operations produces incorrect results, a broken isomorphism where data representations are not reliably convertible, or the complete

absence of necessary morphisms to handle environmental conditions.

2.1. Tracing the Data Flow: Mapping Code to the Model

The current implementation appears to follow a sequential, and somewhat naive, composition of the required operations. A user selects a video, applies edits, and upon naming the move, a series of asynchronous callbacks are triggered to process the video, save it to the Photos library, and then update the CoreData database. The lack of a central coordinator or state management leads to race conditions and unhandled failure paths, which manifest as the bugs reported by the user.

2.2. The Rotation-Trim Anomaly (Non-Commutative Composition)

Observed Behavior: Videos are saved with incorrect orientation or are improperly cropped, particularly when both trimming and rotation are applied. The final asset in the Photos library does not match the preview shown to the user in the editing interface.

Categorical Analogy: The composition of transformations within the `exportWithEdits` morphism is non-commutative and incorrect. The function that generates the preview, let's

call it `preview`, produces a different visual result than the function that

generates the final file, . This violates the commutative property required for a WYSIWYG experience.

Root Cause Analysis: The fundamental error lies in the application of geometric transforms using AVFoundation. The provided logs suggest that a rotation is being applied via an AVMutableVideoCompositionLayerInstruction. However, the implementation fails to account for the coordinate system of the video composition. As detailed in Apple's own technical Q&A and confirmed by numerous developer reports, CGAffineTransform rotations are applied relative to the composition's origin, which is the top-left corner (0,0), not the center of the video frame.⁷

When a simple rotation matrix, such as one created by CGAffineTransformMakeRotation, is applied to a video layer, it rotates the entire frame around this top-left point. For a 90-degree clockwise rotation of a portrait video, this swings the top edge of the video out of the visible frame to the right, while the left edge swings up. The AVAssetExportSession then renders this transformed composition, resulting in a final video that appears cropped or shows black bars, as most of the visual content has been rotated "off-screen."

The correct approach requires a composite transformation. First, the rotation is applied. Second, a translation must be applied to move the rotated frame back into the visible bounds. The required translation vector is a function of the video track's naturalSize and the angle of rotation. For example, after a 90-degree clockwise rotation, the origin of the rotated frame is now at what was the top-right corner of the original frame in the composition space. A translation is needed to shift the frame left by its original height. The current implementation is missing this critical translation component, demonstrating a failure to preserve the visual integrity of the asset through the exportWithEdits morphism.⁹

2.3. Album Management Idempotency Failure

Observed Behavior: The application creates duplicate "BreakDex" albums in the user's

Photos library. In some cases, these albums may be empty.

Categorical Analogy: The `saveToPhotos` morphism is not idempotent. An idempotent operation is one that can be applied multiple times without changing the result beyond the initial application. In this case, repeatedly executing the save logic produces multiple side effects (new albums), whereas it should always converge to a single, correct state (one "BreakDex" album containing all saved videos).

Root Cause Analysis: This behavior is a classic symptom of a race condition in a "read-then-write" pattern performed without proper transactional integrity. The current logic likely performs these steps sequentially:

1. Fetch all albums to check if one named "BreakDex" exists.
2. If it does not exist, create a new album named "BreakDex".
3. Add the new video asset to the album.

If two save operations are initiated in quick succession, both may execute step 1 and find no existing album. Both will then proceed to execute step 2, resulting in the creation of two separate albums. The PhotoKit framework provides a specific mechanism to prevent this: the `PHPhotoLibrary.shared().performChanges` block. This block acts as a transaction, ensuring that all changes requested within it are performed atomically. The correct implementation must perform the entire find-or-create-and-add sequence within a *single* `performChanges` call. By failing to do so, the system is not treating its interaction with the external Photos library as an atomic transaction, leading to an inconsistent state.⁴

2.4. The Broken Isomorphism (Data Source Decoupling)

Observed Behavior: A move is successfully saved from the user's perspective, but the associated video does not appear in the `MoveDetailView`, or the application crashes when attempting to load the video for playback.

Categorical Analogy: The intended isomorphism between the `LibraryPHAsset` (the asset in the Photos library) and the `CoreDataRecord` (the app's database entry) is broken. An isomorphism is a structure-preserving, invertible map; one should be able to reliably get from the database record to the correct photo asset, and vice-versa. The current implementation fails this test because the reference stored in CoreData does not reliably map back to a valid, accessible asset.

Root Cause Analysis: This failure stems from a fundamental misunderstanding of data ownership and lifecycle on iOS. The application is likely storing a transient or incorrect identifier in its CoreData database. A common mistake is to save the file path of the

ProcessedVideoFile from the app's temporary directory. This file is ephemeral; the operating system is free to delete it at any time to reclaim space, immediately breaking the link and leaving the CoreData record pointing to a non-existent location.

The absolute source of truth for user media is the Photos library. Therefore, the only stable and correct reference to store is the localIdentifier of the PHAsset. This is a unique, persistent string generated by PhotoKit when the asset is first created in the library.¹² The correct data flow requires the application to successfully complete the

saveToPhotos morphism, retrieve the localIdentifier of the newly created LibraryPHAsset from the completion handler of the performChanges block, and only then execute the commitToDatabase morphism, storing this permanent identifier. By storing a reference to a temporary file it created, the app incorrectly assumes ownership of the media. The correct architectural model is to cede ownership to the Photos library and operate as a client holding a stable, system-provided key (localIdentifier) to that asset. This decouples the app's internal data from the asset's physical storage, making the system resilient to OS-level file management, iCloud synchronization, and device backups.⁶

2.5. Unhandled Morphisms (Environmental Blind Spots)

The current system's architecture exhibits significant blind spots by failing to account for common environmental conditions. These omissions are equivalent to missing morphisms in our categorical model, representing entire classes of state transitions that the system cannot handle.

2.5.1. The iCloud Asset Path

Failure: The application freezes, becomes unresponsive, or fails when a user selects a video that is stored only in iCloud and has not been downloaded to the device.

Missing Morphism: . The system incorrectly assumes that the applyEdits morphism is always a simple, synchronous operation.

Analysis: When a user's photo library is optimized for storage, many assets, especially larger video files, may exist only as low-resolution thumbnails on the device. To process such an asset, it must first be downloaded. The PhotoKit framework provides APIs to manage this process. The application must first check if the asset data is available locally. This can be done by attempting to request the asset (e.g., via `PHImageManager.default().requestAVAssetForVideo`) with the `networkAccessAllowed` option of `PHVideoRequestOptions` set to false. If this request fails or returns a nil asset, it indicates the data is in iCloud.¹³ The application must then make a second request with

`networkAccessAllowed` set to true. This second request is asynchronous and should provide progress feedback to the user via the `progressHandler` property, which receives updates on the download's progress. The current implementation lacks this entire detection and download flow, leading to failure whenever it encounters a non-local asset.¹⁴

2.5.2. The Background Execution Path

Failure: The export process for longer videos fails if the user switches to another application or locks their device screen before the export completes.

Missing Morphism: . The system does not request the necessary permissions to continue execution in the background.

Analysis: An AVAssetExportSession can take a considerable amount of time, from seconds to many minutes, depending on the video's length, resolution, and the complexity of the edits. Standard iOS application lifecycle rules will suspend an app shortly after it moves to the background. If the export is still in progress, the system will terminate it. To prevent this, the application must explicitly request extended background execution time. The modern BackgroundTasks framework is designed for this purpose. For a user-initiated, potentially long-running task like a video export, the BGContinuedProcessingTask is the ideal mechanism. This task is initiated while the app is in the foreground and signals to the system that the work is critical and should be allowed to continue for a reasonable period if the app is backgrounded. The documentation explicitly lists "Exporting video in a film-editing app" as a primary use case for this API.¹⁷ The absence of any background task management means the current implementation is only reliable for very short export operations.

2.5.3. The Limited Library Path

Failure: The application may not be able to access any videos, or may behave unpredictably for users who have granted "Select Photos..." (limited library) permission, a feature introduced in iOS 14.

Missing Logic: The system does not correctly query for or handle the .limited authorization status.

Analysis: Since iOS 14, users have granular control over their photo library access and can choose to grant an app access to only a specific subset of their photos and videos.¹⁹ When an app is in this

.limited state, standard fetch requests like PHAsset.fetchAssets will only return assets from

within that user-selected subset.²⁰ The current application likely assumes it always has full

.authorized access. A robust implementation must check for the .limited status using `PHPhotoLibrary.authorizationStatus(for:.readWrite)`. If the status is limited, the app should function correctly with the subset of available assets. Furthermore, it should provide a clear user interface element, such as a "Manage Photo Access" button, that programmatically presents the limited library selection interface via `PHPhotoLibrary.shared().presentLimitedLibraryPicker(from:)`. This empowers the user to grant the app access to additional assets without having to navigate deep into the system's Settings app.¹⁹

Table 1: Failure Point Analysis

Failure Point	Observed Behavior	Categorical Analogy	Root Cause	Key Supporting Documentation
Incorrect Video Rotation	Saved video is rotated 90 degrees and cropped or has black bars.	Non-commutative composition of rotation and translation transforms.	Applying <code>CGAffineTransformMakeRotation</code> without a corresponding <code>CGAffineTransformMakeTranslation</code> to recenter the frame post-rotation.	⁷
Duplicate Album Creation	Multiple "BreakDex" albums appear in the Photos app.	<code>saveToPhotos</code> morphism is not idempotent.	A read-then-write race condition; failure to use a single, atomic <code>PHPhotoLibrary.performChanges</code> transaction for the	⁴

			find-or-create operation.	
Broken Data Reference	Saved video is not displayed in the app; potential for crashes.	The mapping between CoreDataRecord and LibraryPHAsset is not an isomorphism.	Storing a transient reference (e.g., temporary file path) in CoreData instead of the permanent localIdentifier of the PHAsset.	2
iCloud Asset Failure	App freezes or fails when selecting a video not stored locally.	Missing downloadFromCloud morphism.	Failure to detect non-local assets and initiate an asynchronous download with networkAccessAllowed and a progressHandler.	13
Background Export Failure	Long video exports are terminated when the app is backgrounded.	Missing executeInBackground morphism.	Failure to request extended execution time using the BackgroundTasks framework (e.g., BGContinuedProcessingTask).	17
Limited Library	App cannot access desired	Incomplete handling of	Failure to check for	19

Incompatibility	videos or presents an empty library.	PHAuthorizationStatus.	.limited photo library access status and provide a mechanism to present the limited library picker.	
------------------------	--------------------------------------	------------------------	---	--

III. Architecting a Fault-Tolerant, Transactional Pipeline

The diagnosis of the system's failures informs the design of a new, robust architecture. The guiding principle of this redesign is to manage the entire video persistence workflow as a single, cohesive, and resilient transaction. This is achieved by introducing a central coordinating entity, implementing correct and structure-preserving data transformations, ensuring atomic interactions with system services, and building in explicit handling for the environmental complexities of the iOS platform.

3.1. The VideoPersistenceManager: A State-Driven Coordinator

To eliminate the scattered, unmanaged asynchronous callbacks that plague the current system, a new class, VideoPersistenceManager, will be introduced. This object will serve as a state machine, encapsulating the entire end-to-end flow and managing the transitions between a well-defined set of states:

- Idle
- CheckingAuthorization
- DownloadingFromCloud(progress: Double)
- ExportingVideo(progress: Double)
- SavingToPhotoLibrary
- SavingToCoreData
- Finished(CoreDataRecord)
- Failed(Error)

By centralizing all logic, state transitions, and error handling within this single coordinator, the

system gains clarity and robustness. The VideoPersistenceManager becomes the sole orchestrator of the morphisms defined in our model. This pattern prevents race conditions and ensures that failures at any stage can be gracefully handled, either by aborting the entire process and cleaning up temporary resources or by implementing a retry mechanism. This approach aligns with best practices for building resilient data pipelines, where a central controller manages the flow of data through various processing stages.²²

3.2. Functorial Video Processing: Preserving Visual Integrity

The exportWithEdits morphism can be modeled as a functor—a structure-preserving map between categories. In this context, it maps the category of AVAssets with editing parameters to the category of processed VideoFiles. To be a valid functor, it must preserve the essential structure, which is the visual representation of the video. The failure to do so (the rotation anomaly) is a failure of the morphism to be truly structure-preserving. The correct implementation will guarantee visual integrity.

Correct Implementation of Rotation and Trimming:

1. **Get Asset Track Properties:** Upon receiving the LocalAVAsset, the first step is to access its primary video AVAssetTrack and inspect its naturalSize and preferredTransform. The preferredTransform is crucial as it contains the video's inherent orientation metadata (e.g., a video shot vertically on an iPhone will have a 90-degree rotation transform embedded in it).
2. **Create the Composition:** An AVMutableComposition is created. The user's desired trimRange is applied by inserting the time range from the source asset's track into the composition's track. This correctly handles the trimming portion of the edit.³
3. **Calculate the Correct Composite Transform:** This is the most critical step to fix the rotation bug. A final CGAffineTransform must be constructed by concatenating multiple transforms in the correct order:
 - Start with the video track's original preferredTransform.
 - Concatenate the user's desired rotation (e.g., an additional 90 degrees).
 - Concatenate the necessary translation to recenter the frame. The logic for this translation depends on the final orientation and the video's naturalSize. For example,

for a portrait video () rotated 90 degrees to

landscape, its new dimensions are . The transform must include a translation on the x-axis by the original height (1920 pixels) to bring it back into the frame.

4. **Build the Composition Instructions:** An `AVMutableVideoCompositionLayerInstruction` is created for the video track, and its transform is set to the final composite transform calculated in the previous step.⁷ This layer instruction is then added to an `AVMutableVideoCompositionInstruction`.
5. **Configure and Run the Export Session:** An `AVMutableVideoComposition` is created and configured with the instruction. Crucially, its `renderSize` property must be set to the dimensions of the video *after* rotation (e.g., swapping width and height for a 90-degree rotation). This video composition is then assigned to the `videoComposition` property of the `AVAssetExportSession`. The export session is then configured with an output URL and initiated.

3.3. Idempotent and Atomic Photo Library Operations

The `saveToPhotos` morphism will be re-implemented to be both atomic and idempotent, ensuring the state of the user's Photos library remains consistent and correct.

Refined Logic Flow:

The entire operation will be encapsulated within a single function that makes one call to `PHPhotoLibrary.shared().performChanges`.

1. Inside the `performChanges` block:

- A fetch request is made for `PHAssetCollections` with the title "BreakDex".
- An if-let statement checks the result of the fetch.
- **If an album is found:** The first `PHAssetCollection` from the result is used to create a `PHAssetCollectionChangeRequest(for:)`.
- **If no album is found:** A `PHAssetCollectionChangeRequest.creationRequestForAssetCollection(withTitle: "BreakDex")` is created.

2. A `PHAssetChangeRequest.creationRequestForAssetFromVideo(atFileURL:)` is created, using the URL of the `ProcessedVideoFile`. This request returns a placeholder object for the new asset.

3. The asset placeholder is then added to the album using the `addAssets(_:)` method on the album change request.

4. In the `completionHandler` of `performChanges`:

- The success boolean is checked.
- If successful, the placeholder object for the created asset is used to retrieve its permanent `localIdentifier`. This identifier is the crucial piece of data that will be passed to the next stage of the pipeline.

This transactional approach, recommended by Apple's frameworks, completely eliminates the race condition and guarantees that an asset is always added to a single, correctly-named album, thus solving the idempotency failure.⁴

3.4. Establishing the `localIdentifier` as the Canonical Reference

The architecture will be modified to enforce that the `PHAsset`'s `localIdentifier` is the one and only canonical reference to the video asset stored within the application's own database.

Refined Data Flow: The `commitToDatabase` morphism is now causally dependent on the successful completion of the `saveToPhotos` morphism. The `localIdentifier` string obtained from the `performChanges` completion handler is the sole asset-related data passed to the CoreData saving function. The `Move` entity in the CoreData model will have a non-optional `assetIdentifier` attribute of type `String`.

This establishes a robust, unidirectional data flow: App Edits $\rightarrow$ Temporary File $\rightarrow$ Photos Library Asset $\rightarrow$ CoreData Reference. When a `MoveDetailView`

needs to display a video, it will:

1. Fetch the Move managed object from CoreData.
2. Retrieve the assetIdentifier string.
3. Use PHAsset.fetchAssets(withLocalIdentifiers:options:) to fetch the specific PHAsset from the Photos library.¹²
4. Use this PHAsset to request an AVPlayerItem for playback.

This ensures the application always loads the definitive "source of truth" from the user's Photos library, making the app's data layer resilient and correctly decoupled from the media storage layer.⁶

3.5. Building for Resilience: Handling the iOS Environment

A robust system must anticipate and handle the complexities of its operating environment. The new architecture will explicitly manage iCloud assets, background execution, and modern privacy controls.

3.5.1. Integrating the iCloud Download Flow

The VideoPersistenceManager will begin its process by checking the state of the selected SourcePHAsset. If a request for the asset with networkAccessAllowed set to false indicates that the asset is not local, the manager will transition to the DownloadingFromCloud state. It will then initiate a new requestAVAssetForVideo call with networkAccessAllowed set to true. The progressHandler of the request options will be configured to receive progress updates (a Double between 0.0 and 1.0). These updates will be published by the manager, allowing the UI to display a determinate progress indicator to the user. The rest of the persistence pipeline will be paused until this download completes successfully or fails, at which point the manager will transition to the ExportingVideo or Failed state, respectively.¹⁴

3.5.2. Ensuring Completion with Background Tasks

Upon user initiation of the save operation, the VideoPersistenceManager will immediately begin a BGContinuedProcessingTask. This API is specifically designed for user-initiated tasks

that need to continue if the app is backgrounded.¹⁸ The entire pipeline—from video export to the final CoreData commit—will execute within the scope of this background task. The task's expiration handler will be configured to gracefully cancel the

AVAssetExportSession if the system decides to terminate the task. Once the entire pipeline successfully completes or fails, the manager will call `setTaskCompleted(success:)` on the background task object to signal its completion to the system. This ensures that even lengthy exports on large video files have the highest possible chance of completing successfully.²⁴

3.5.3. Gracefully Handling Limited Library Access

The user interface layer responsible for video selection will be updated to be aware of the `.limited` photo library authorization status. Before presenting a picker, it will query the status using `PHPhotoLibrary.authorizationStatus(for:.readWrite)`.

- If the status is `.authorized`, the flow proceeds as normal.
- If the status is `.limited`, the UI can present the standard picker, which will be automatically filtered to the user's allowed selection. In addition, a separate, clearly labeled button such as "Manage Photo Access" or "Select More Videos" will be displayed. Tapping this button will call `PHPhotoLibrary.shared().presentLimitedLibraryPicker(from:)`, which presents the system UI for managing the app's photo selection. This provides a seamless, in-app workflow for the user to grant access to more assets without interrupting their task.¹⁹

IV. Refined Implementation and Verification

This section provides concrete implementation guidance for the proposed architecture and outlines a strategy for verifying its correctness. The goal is to provide actionable code patterns and a testing methodology that closes the loop on the formal model, ensuring the final system is provably robust.

4.1. Implementation Snippets for Key Components

The following code skeletons illustrate the structure of the key services in the redesigned

architecture.

VideoPersistenceManager.swift

This class acts as the central state machine and coordinator for the entire process.

Swift

```
import Foundation
import BackgroundTasks
import Photos

class VideoPersistenceManager {
    // State management and UI updates would be handled via Combine or delegation
    enum State {
        case idle, checkingAuthorization, downloading(Double), exporting(Double)
        case savingToLibrary, savingToCoreData, finished, failed(Error)
    }

    private var backgroundTask: BGTask?

    func save(move: MoveCreationRequest) {
        // 1. Begin a background task to ensure completion
        registerAndBeginBackgroundTask()

        // 2. Start the state machine pipeline
        //... check permissions, handle iCloud download, process video,
        // save to library, save to CoreData...
    }

    private func registerAndBeginBackgroundTask() {
        // Register the task identifier in Info.plist
        // Use BGContinuedProcessingTask for user-initiated work
        //... implementation details...
    }

    //... other private methods for each step of the pipeline...

    private func cleanup(withError error: Error? = nil) {
        // Clean up temporary files
        // End the background task
        backgroundTask?.setTaskCompleted(success: error == nil)
    }
}
```

```
}  
}
```

VideoProcessor.swift

This service encapsulates all AVFoundation logic, ensuring transformations are applied correctly.

Swift

```
import AVFoundation
```

```
struct VideoEdits {  
    let trimRange: CMTimeRange  
    let rotationAngle: CGFloat // in degrees  
}
```

```
class VideoProcessor {  
    func processVideo(asset: AVAsset, edits: VideoEdits, completion: @escaping (Result<URL, Error>) ->  
    Void) {  
        // 1. Create AVMutableComposition and insert trimmed time range  
        let composition = AVMutableComposition()  
        //... logic from...[3]  
  
        // 2. Get video track and its properties  
        guard let videoTrack = asset.tracks(withMediaType:.video).first else { /* handle error */ return  
    }  
        let naturalSize = videoTrack.naturalSize  
        let preferredTransform = videoTrack.preferredTransform  
  
        // 3. Calculate the correct composite transform for rotation + translation  
        var finalTransform = preferredTransform // Start with original orientation  
        //... apply user rotation and calculate translation based on naturalSize...  
        // This logic is critical and must correctly handle the geometry as per.[8]  
  
        // 4. Create layer instruction and video composition  
        let layerInstruction = AVMutableVideoCompositionLayerInstruction(assetTrack:  
composition.tracks(withMediaType:.video).first!)  
        layerInstruction.setTransform(finalTransform, at:.zero)  
  
        let mainInstruction = AVMutableVideoCompositionInstruction()
```

```

mainInstruction.timeRange = edits.trimRange
mainInstruction.layerInstructions = [layerInstruction]

let videoComposition = AVMutableVideoComposition()
videoComposition.instructions = [mainInstruction]
videoComposition.frameDuration = CMTime(value: 1, timescale: 30)
// Set renderSize to the post-rotation dimensions
videoComposition.renderSize = /* calculated size after rotation */

// 5. Configure and run AVAssetExportSession
guard let exportSession = AVAssetExportSession(asset: composition, presetName:
AVAssetExportPresetHighestQuality) else { /* handle error */ return }
exportSession.videoComposition = videoComposition
//... configure output URL, file type, and run export...
}
}

```

PhotoKitService.swift

This service handles all interactions with the Photos library in an atomic fashion.

Swift

```

import Photos

class PhotoKitService {
    private let albumName = "BreakDex"

    func saveVideo(from fileURL: URL, completion: @escaping (Result<String, Error>) -> Void) {
        var albumPlaceholder: PHObjectPlaceholder?
        var assetPlaceholder: PHObjectPlaceholder?

        PHPhotoLibrary.shared().performChanges({
            // 1. Create change request for the asset
            let assetRequest =
PHAssetChangeRequest.creationRequestForAssetFromVideo(atFileURL: fileURL)
            assetPlaceholder = assetRequest?.placeholderForCreatedAsset

            // 2. Find or create the album
            let fetchOptions = PHFetchOptions()
            fetchOptions.predicate = NSPredicate(format: "title = %@", self.albumName)

```

```

    let collections = PHAssetCollection.fetchAssetCollections(with:.album, subtype:.any,
options: fetchOptions)

    let albumChangeRequest: PHAssetCollectionChangeRequest?
    if let album = collections.firstObject {
        albumChangeRequest = PHAssetCollectionChangeRequest(for: album)
    } else {
        albumChangeRequest =
PHAssetCollectionChangeRequest.creationRequestForAssetCollection(withTitle:
self.albumName)
        albumPlaceholder = albumChangeRequest?.placeholderForCreatedAssetCollection
    }

    // 3. Add asset to album
    if let assetPlaceholder = assetPlaceholder {
        albumChangeRequest?.addAssets([assetPlaceholder] as NSArray)
    }
}) { success, error in
    if success, let localIdentifier = assetPlaceholder?.localIdentifier {
        completion(.success(localIdentifier))
    } else {
        completion(.failure(error ?? /* custom error */) )
    }
}
}
}
}
}

```

4.2. Verification Through Commutativity: A Testing Strategy

The correctness of the new architecture can be verified by ensuring that the system's diagram commutes—that the final state is always the correct and expected one.

- **Unit Tests:**

- **VideoProcessor:** Write tests for the transform calculation logic. Provide a matrix of inputs (e.g., portrait/landscape video dimensions, 0/90/180/270 degree rotations) and assert that the calculated CGAffineTransform and renderSize are correct.
- **PhotoKitService:** Mocking PHPhotoLibrary is complex. However, integration tests can verify idempotency by calling the saveVideo method multiple times in quick succession and asserting that only one album is created and all assets are added to it.

- **Integration Tests:**

- **End-to-End Local Video Test:** Create a test that runs the full VideoPersistenceManager pipeline using a video asset bundled with the test target. The test will succeed if it completes without error and the localIdentifier stored in the mock CoreData context correctly fetches a PHAsset whose properties (e.g., duration) match the applied trim.
- **End-to-End iCloud Video Test:** This is more complex but can be simulated. The test can use a real iCloud asset and verify that the Downloading state is entered and that the completionHandler is called.
- **Background Execution Test:** Manually trigger the save process and immediately background the app. Use Xcode's debugging tools to verify that the BGContinuedProcessingTask is active and that the export continues to make progress.

The ultimate commutativity check is a visual one performed during manual QA: for any combination of trim and rotation, the video saved in the Photos app must look identical to the final preview in the BreakDex app before saving.

4.3. Addressing Blind Spots and Future Considerations

Even with a robust architecture, potential edge cases and areas for improvement remain.

- **Permission Changes Mid-Session:** A user can change photo library permissions in the Settings app while the app is running or even while an operation is in progress. The VideoPersistenceManager should register as an observer for PHPhotoLibrary.photoLibraryDidChange notifications. If a change notification arrives that indicates a change in authorization status, any in-flight operations should be gracefully cancelled.
- **Temporary File Management:** The ProcessedVideoFile is a temporary artifact that can consume significant storage. The VideoPersistenceManager must implement robust cleanup logic. A defer block or similar mechanism within the main save function should be used to guarantee that the temporary file is deleted upon both successful completion and any failure along the pipeline.
- **Performance Optimization:** While correctness is the primary goal, export performance is also important. The AVAssetExportSession offers several properties for tuning. The presetName should be chosen carefully; AVAssetExportPresetHighestQuality provides the best result but may be slow. Other presets might offer a better balance of speed and quality. Additionally, shouldOptimizeForNetworkUse can be set to true if the videos are intended for streaming, as this rearranges the file structure for progressive downloads.²⁵
- **User Experience:** For long-running operations, clear and persistent feedback is

non-negotiable. The VideoPersistenceManager's state should be bound to the UI to show determinate progress bars for both iCloud downloads and video exports. Since the operation can continue in the background, the use of UserNotifications to inform the user of successful completion or failure is highly recommended.

Works cited

1. PhotoKit | Apple Developer Documentation, accessed October 3, 2025, <https://developer.apple.com/documentation/photokit>
2. iphone - Storing photos and videos in Core Data? - Stack Overflow, accessed October 3, 2025, <https://stackoverflow.com/questions/4545527/storing-photos-and-videos-in-core-data>
3. Trim video using AVFoundation in Swift · GitHub, accessed October 3, 2025, <https://gist.github.com/b8c5f8eafe0605a38692>
4. ios - iPhone - How to create a custom album and give custom names to photos in camera roll programmatically? - Stack Overflow, accessed October 3, 2025, <https://stackoverflow.com/questions/12152430/iphone-how-to-create-a-custom-album-and-give-custom-names-to-photos-in-camera>
5. swift - Save video to a custom Album using Photos Framework in ..., accessed October 3, 2025, <https://stackoverflow.com/questions/44669178/save-video-to-a-custom-album-using-photos-framework-in-ios>
6. NSManagedObjectID and PersistentIdentifier - Mastering Data Identifiers in Core Data and SwiftData - Fatbobman's Blog, accessed October 3, 2025, <https://fatbobman.com/en/posts/nsmanagedobjectid-and-persistentidentifier/>
7. Technical Q&A QA1744: Setting the orientation of video with AV Foundation, accessed October 3, 2025, https://developer.apple.com/library/archive/qa/qa1744/_index.html
8. Rotating Video with AVMutableVideoCompositionLayerInstruction - Stack Overflow, accessed October 3, 2025, <https://stackoverflow.com/questions/5187789/rotating-video-with-avmutablevideocompositionlayerinstruction>
9. Video rotated after applying AVVideoComposition - Stack Overflow, accessed October 3, 2025, <https://stackoverflow.com/questions/44911802/video-rotated-after-applying-avvideocomposition>
10. Objective C - Rotate a video file with AVMutableVideoCompositionLayerInstruction - GitHub Gist, accessed October 3, 2025, <https://gist.github.com/nguyen-tam/1eafe29f6b67c0e1b6fb23deda65f932>
11. Browsing and Modifying Photo Albums | Apple Developer Documentation, accessed October 3, 2025, <https://developer.apple.com/documentation/photokit/browsing-and-modifying-photo-albums>

12. Fetching Assets | Apple Developer Documentation, accessed October 3, 2025, <https://developer.apple.com/documentation/photokit/fetching-assets>
13. ios - How to identify if a PHAsset is not completely downloaded from ..., accessed October 3, 2025, <https://stackoverflow.com/questions/28482275/how-to-identify-if-a-phasset-is-not-completely-downloaded-from-icloud-so-i-need>
14. PHAssetVideoProgressHandler | Apple Developer Documentation, accessed October 3, 2025, <https://developer.apple.com/documentation/photos/phassetvideoprogresshandler>
15. Handling iCloud Assets - Swiftjective-C, accessed October 3, 2025, <https://www.swiftjective.com/icloud-photo-handling/>
16. PHVideoRequestOptions' progressHandler not called - Stack Overflow, accessed October 3, 2025, <https://stackoverflow.com/questions/34962522/phvideorequestoptions-progresshandler-not-called>
17. Choosing Background Strategies for Your App | Apple Developer ..., accessed October 3, 2025, <https://developer.apple.com/documentation/backgroundtasks/choosing-background-strategies-for-your-app>
18. Performing long-running tasks on iOS and iPadOS | Apple Developer Documentation, accessed October 3, 2025, <https://developer.apple.com/documentation/BackgroundTasks/performing-long-running-tasks-on-ios-and-ipados>
19. Delivering an Enhanced Privacy Experience in Your Photos App ..., accessed October 3, 2025, <https://developer.apple.com/documentation/PhotoKit/delivering-an-enhanced-privacy-experience-in-your-photos-app>
20. How to handle limited access - Photo Library - Blog, accessed October 3, 2025, <https://blog.appmetry.com/ios/all-about-limited-access-to-photo-library/>
21. Handle the Limited Photos Library in your app | Documentation - WWDC Notes, accessed October 3, 2025, <https://wwdcnotes.com/documentation/wwdcnotes/wwdc20-10641-handle-the-limited-photos-library-in-your-app/>
22. Swift: Expedited Failure Recovery for Large-Scale DNN Training - IEEE Computer Society, accessed October 3, 2025, <https://www.computer.org/csdl/journal/td/2024/09/10601499/1YGs8fX32Cc>
23. Building a Resilient Data Infrastructure: Best Practices for Fault-Tolerant Systems, accessed October 3, 2025, <https://abrkljac.medium.com/building-a-resilient-data-infrastructure-best-practices-for-fault-tolerant-systems-562587e136a>
24. Using background tasks to update your app | Apple Developer ..., accessed October 3, 2025, <https://developer.apple.com/documentation/uikit/using-background-tasks-to-update-your-app>

25. shouldOptimizeForNetworkUse | Apple Developer Documentation, accessed October 3, 2025,
<https://developer.apple.com/documentation/avfoundation/avassetexportsession/shouldoptimizefornetworkuse>
26. AVAssetExportSession | Apple Developer Documentation, accessed October 3, 2025,
<https://developer.apple.com/documentation/avfoundation/avassetexportsession>
27. Exporting video to alternative formats | Apple Developer Documentation, accessed October 3, 2025,
<https://developer.apple.com/documentation/avfoundation/exporting-video-to-alternative-formats>